

Proxima Compass Base - API Manual

1.9

The following REST APIs can be called at port 1984 at the same PC IP address where Proxima Compass Base has been installed. Where not specified the media type is application/json. If you are connected to the robot by a LAN or VPN connection, you should use the IP of the robot to access the API.

/configuration	3
POST	3
GET	4
/custom-waypoint-mission	5
POST	5
/global-path	6
GET	6
/joy-input	6
POST	6
/laser/{step}	7
GET	7
/localize-pose	7
POST	8
/localize-waypoint	8
POST	8
/login	9
POST	9
/logout	9
POST	9
/map-info	10
GET	10
/map-pose	10
GET	10
/mapping	11
POST	11
GET	11
/mission	12
POST	12
/move	13
POST	13
/navigation-map	13
POST	13
GET	14
/navigation-mode	15
POST	15
/obstacles	16
GET	16
/raw-scans	16
GET	16
/raw-scans/{name}	17
GET	17

DELETE	17
/robot-status	18
GET	18
/set-parameters	18
POST	18
/system-status	19
POST	19
GET	19
/system-status/{status}	20
GET	20
Idle	20
Configuration	20
PreMapping or Mapping	20
PreNavigation or Navigation	21
Test	21
/system-status/substatus	21
GET	21
/software-version	22
GET	22
/test	22
POST	22
/velocities	23
GET	23
/world-pose	23
GET	23
Appendix	25
State machine	25
Endpoints vs State	26
Token and timeout	27
Test and configuration combinations	27

/configuration

Set or get the main parameters for the robot. See User Guide manual for specific variables to add/remove depending on kinematics type, motion protocol type and control type. [kinematics] in motion objects will be one of the feasible ones described in the kinematics section, field type.

POST

parameters

- no parameters

request body

- general: object
 - robot_name
- connections: object
 - ip: string
 - cidr: number
- kinematics: object
 - type:string [diff_drive, omnidirectional]
 - wheel_radius: number (m)
 - wheelbase: number (m) - omnidirectional only
 - reduction: number
 - axle_length: number (m)
 - footprint_a: number (m)
 - footprint_b: number (m)
 - footprint_c_d: number (m)
 - vel_lin_max: number (m/s)
 - vel_ang_max: number (rad/s)
 - acc_lin_max: number (m/s²)
 - acc_ang_max: number (rad/s²)
 - dec_lin_max: number (m/s²)
 - acc_lat_max: number (m/s²)
 - align_acc_ang_max: number (rad/s²),
 - align_vel_ang_max: number (rad/s),
 - align_vel_ang_min: number (rad/s),
 - padding: number (m),
 - obstacle_stop_distance: number (m),
 - slowdown_area_vel_max: number (m/s)
- motion: object
 - type: string [opcua, ros2]
 - server_opcua_ip: string (only if type = opcua)
 - server_opcua_port: string (only if type = opcua)
 - [kinematics]: Object
 - rpm_cmd_left: string - diff_drive, opcua
 - rpm_cmd_right: string - diff_drive, opcua
 - rpm_status_left: string - diff_drive, opcua
 - rpm_status_right: string - diff_drive, opcua
 - rpm_cmd_front_left: string - omnidirectional, opcua
 - rpm_cmd_front_right: string - omnidirectional, opcua
 - rpm_cmd_rear_left: string - omnidirectional, opcua
 - rpm_cmd_rear_right: string - omnidirectional, opcua
 - rpm_status_front_left: string - omnidirectional, opcua
 - rpm_status_front_right: string - omnidirectional, opcua
 - rpm_status_rear_left: string - omnidirectional, opcua
 - rpm_status_rear_right: string - omnidirectional, opcua
 - twist_cmd_topic: string - ros2

- odometry_status_topic: string - ros2
 - safety: string - opcua
 - safety_topic: string - ros2
 - front_left_distance: string - opcua,
 - front_right_distance: string - opcua,
 - rear_left_distance: string - opcua,
 - rear_right_distance: string - opcua
- sensors: array of objects
 - type: string [datalogic, pf_r2000, sick_scanner, ros2_scanner, pilz_psenscan]
 - x: number
 - y: number
 - z: number
 - yaw: number (rad)
 - ip: string - datalogic, sick_scanner, pilz_psenscan, pf_r2000
 - min_ang: number (rad) - datalogic, sick_scanner, pilz_psenscan
 - max_ang: number (rad) - datalogic, sick_scanner, pilz_psenscan
 - topic: string - ros2_scanner
 - frame: string - ros2_scanner
 - is_master: boolean - pilz_psenscan
 - index: number - pilz_psenscan

responses (code, description)

- 200, Successful operation
- 400, Invalid request or no sensors added or unable to configure the network
- 500, Unable to connect to server or database

GET

parameters

- no parameters

request body

- no request body

response

- 200, json formatted string

example value

```
{
  "content": {
    "general": {
      "robot_name": "proximino"
    }
  },
  "connections": {
    "ip": "localhost",
    "cidr": 24
  },
  "kinematics": {
    "type": "diff_drive",
    "wheel_radius": 0.0782,
    "reduction": 1.0,
    "axle_length": 0.435,
    "footprint_a": 0.28,
    "footprint_b": 0.28,
    "footprint_c_d": 0.25,
    "padding": 0.05,
    "vel_lin_max": 0.3,
    "acc_lin_max": 0.1,
  }
}
```

```

    "vel_ang_max": 0.5,
    "acc_ang_max": 0.35,
    "acc_lat_max": 0.04,
    "dec_lin_max": 0.075,
    "align_acc_ang_max": 0.4,
    "align_vel_ang_max": 0.5,
    "align_vel_ang_min": 0.07,
    "obstacle_stop_distance": 0.7
  },
  "motion": {
    "type": "opcua",
    "server_opcua_ip": "localhost",
    "server_opcua_port": 4840,
    "safety": "Objects/COMPASS_BASE/Safety",
    "front_left_distance": "Objects/COMPASS_BASE/FrontLeftDistance",
    "front_right_distance": "Objects/COMPASS_BASE/FrontRightDistance",
    "rear_left_distance": "Objects/COMPASS_BASE/RearLeftDistance",
    "rear_right_distance": "Objects/COMPASS_BASE/RearRightDistance",
    "diff_drive": {
      "rpm_cmd_left": "Objects/COMPASS_BASE/RpmLeftReq",
      "rpm_cmd_right": "Objects/COMPASS_BASE/RpmRightReq",
      "rpm_status_left": "Objects/COMPASS_BASE/RpmLeftStatus",
      "rpm_status_right": "Objects/COMPASS_BASE/RpmRightStatus"
    }
  },
  "sensors": [
    {
      "type": "datalogic_sentinel",
      "ip": "localhost",
      "x": 0.212,
      "y": 0.167,
      "z": 0.077,
      "yaw": 45.0001,
      "min_ang": -129.5,
      "max_ang": 107.5
    }
  ],
}

```

/custom-waypoint-mission

Set a custom waypoint, i.e., directly specifying the desired 2-D position (x, y, yaw angle).

POST

parameters

- no parameters

request body

- x: number (x-position in meters)
- y: number (y-position in meters)
- yaw: number (orientation in radians)
- map: string (name of the map)

example value

```
{
```

```
    "x": 2.0
    "y": 1.0
    "yaw": 0.0
    "map": "map_name"
  }
responses
  • 200, Successful operation
  • 400, Bad request
  • 404, Not found
  • 500, Internal server error
example value
  • 200, "Success"
  • 400, "Invalid data type"
```

/global-path

Get the path the robot is following

GET

parameters

- no parameters

request body

- no request body

response

- 200, successful operation

example value

"content":

```
{
  "path": [
    {
      "x": number (meters),
      "y": number (meters),
    },
    .....
    {
      "x": number (meters),
      "y": number (meters),
    }
  ]
}
```

/joy-input

Read the input provided by the joystick.

POST

parameters

- no parameters

request body

- x: number, x-position of the joystick

- y: number, y-position of the joystick

example value

```
{
  "x": 0.5,
  "y": 0.7
}
```

responses

- 200, Successful operation
- 400, Bad request

example value

- 200, "Success"
- 400, "ERROR: Cannot find x or y fields in post request"

/laser/{step}

Retrieve the laser points from the localization scan.

The step parameter [1, inf) allows to retrieve one laser point every *step* number of points of the total scan.

E.g., /laser/10 considers one point every 10 points of the total scan (therefore, if the total points of the scan are 3600, an array of 360 pts will be retrieved).

GET

parameters

- step

request body

- no request body

response

- 200, successful operation

example value

```
"content":
{
  "points": [
    {
      "x": number (meters),
      "y": number (meters),
      "intensity": number (adimensional)
    },
    .....
    {
      "x": number (meters),
      "y": number (meters),
      "intensity": number (adimensional)
    }
  ]
}
```

/localize-pose

Localize the robot in map *map* with a defined pose.

POST

parameters

- no parameters

request body

- map: string (name of the map)
- x: number (position in the x-axes in meters)
- y: number (position in the y-axes in meters)
- yaw: number (orientation in radians)

example value

```
{
  "map": "awesome_map",
  "x": 1.0,
  "y": 2.3,
  "yaw": 0.0
}
```

responses

- 200, Successful operation
- 400, Bad request
- 500, Internal error

example value

- 200, "Success"
- 400, "Invalid localization request"
- 500, "Error during localization"

/localize-waypoint

Localize the robot in map *map* using a predefined waypoint.

POST

parameters

- no parameters

request body

- map: string (name of the map)
- waypoint: string (name of the waypoint)

example value

```
{
  "map": "awesome_map",
  "waypoint": "0"
}
```

responses

- 200, Successful operation
- 400, Bad request
- 404, Waypoint not found
- 500, Internal error

example value

- 200, "Success"
- 400, "Invalid localization request"
- 404, "Error, waypoint not found"
- 500, "Error during localization"

/login

Do a login operation

POST

parameters

- no parameters

request body

- username: string
- password: string

example value

```
{
  "username": "username",
  "password": "password"
}
```

response

- 200, successful operation
 - it will return also the token generated from backend; the token must be attached at each successive request to the backend in the *Authorization* header field; logout included; see appendix for details
- 400, invalid request
- 401, no more authorized
- 403, forbidden someone else is already logged in
- 403, username not registered
- 403, username or password not valid
- 500, internal database error

example value

- 403, "ERROR: Username or password are not valid"

/logout

Do a log out operation

POST

parameters

- no parameters

request body

- no request body

response

- 200, Successful operation
 - the token previously stored should be removed
- 400, Invalid request
- 500, internal error

example value

- 200, "Success"
- 400, "ERROR: Cannot find username to logout"
- 500, "ERROR: Logout failed"

/map-info

Retrieve navigation map only information such as: width of the image; height of the image; resolution (that indicates how many meters represent in the world a single pixel in the image); origin_x and origin_y that represent the origin coordinates of the map with respect to the world.

GET

parameters

- no parameters

request body

- no request body

responses

- 200, json formatted string
- 404, File not found

example value

```
{
  "content":
  {
    "width": number (pixel)
    "height": number (pixel)
    "resolution": number (meters)
    "origin_x": number (meters)
    "origin_y": number (meters)
  }
}
```

/map-pose

Take the robot position in map frame

GET

parameters

- no parameters

request body

- no request body

responses

- 200, json formatted string
- 404, Failed to open png

example value

```
{
  "content":
  {
    "pose":
    {
      "x": number (meters)
      "y": number (meters)
      "yaw": number (radians)
    }
    "map_info":
```

```

    {
        "width": number (pixel)
        "height": number (pixel)
        "resolution": number (adimensional)
        "origin_x": number (meters)
        "origin_y": number (meters)
    }
}

```

/mapping

Start SLAM mapping, stop it or get mapping data

POST

parameters

- no parameters

request body

- action: string {start-mapping, stop-mapping}

example value

```

{
    "action": "start-mapping"
}

```

responses (code, description)

- 200, successful operation
- 400, wrong requested action
- 500, error in mapping start/stop
- 507, no more space to store map

example value

- 400, "Invalid mapping request"

GET

parameters

- no parameters

request body

- no request body

responses

- 200, json formatted string
- 404, Failed to open png

example value

```

{
    "content":
    {
        "scan": string //(image encoded in base64 format)
        "map_info": {
            "width": number (pixel)
            "height": number (pixel)
            "resolution": number (meters)
            "origin_x": number (meters)

```

```

        "origin_y": number (meters)
    }
}

```

/mission

Handle mission operations such as start, abort, pause and resume

POST

parameters

- no parameters

request body

- action: string (type of the action to perform [start-mission | stop-mission pause | resume])
- waypoints: array of objects (useful only for "start-mission" action)
 - name: string (waypoint name)
 - radius: number (double, interpolation radius, the precision for positioning, in meters)
- map: string (name of the map; useful only for "start-mission" action)

example value

```

{
    "action": "start-mission",
    "waypoints": [
        {
            name: "1",
            radius: 0.2
        },
        ..... // other waypoints
        {
            name: "0",
            radius: 0.15
        },
    ]
    "map": "navigation_map"
}

```

or

```

{
    "action": "pause"
}

```

or

```

{
    "action": "resume"
}

```

or

```

{
    "action": "stop-mission"
}

```

responses

- 200, Successful operation
- 400, Bad request
- 500, Internal error

example value

- 200, "Success"
- 400, "Error in parse action request"
- 404, "Error, waypoint not found"
- 500, "[ERROR] Connection to db failed: retry operation"

/move

Handle relative movement with respect to the current position

POST

parameters

- no parameters

request body

- type: string (type of the movement ["linear" | "angular"])
- distance: number (double, meter for linear movement, radians for angular movement)
- velocity: number (double, m/s for linear movement, rad/s for angular movement)

example value

```
{
  "type": "linear",
  "distance": 1.0,
  "velocity": 0.2
}
```

responses

- 200, Successful operation
- 400, Bad request
- 500, Internal error

example value

- 200, "Success"
- 400, "Invalid distance request format"
- 500, "Unable to execute relative move action"

ATTENZIONE: nel caso in cui si voglia abortire la missione in corso si deve inviare richiesta POST sull'endpoint /mission del tipo:

```
{
  "action": "stop-mission"
}
```

/navigation-map

Save or load navigation map data (i.e., the map and the waypoints of interest)

POST

parameters

- no parameters

request body

- waypoints array of objects
 - name: string (waypoint name)
 - x: number (x-position in meters)
 - y: number (y-position in meters)

- yaw: number (orientation in radians)
- map: string (img encoded in base64 format)

example value

```
{
  "waypoints": [
    {
      "index": 0
      "name": "1",
      "x": 1.04,
      "y": 4.23,
      "yaw": 1.57,
      "type": "target"
    },
    ..... // other waypoints
  ],
  "areas": [
    {
      index: 0
      type: slowdown_areas,
      name: area_1
      points: [{x:0,y:0,index:0}...]
    }
  ]
  map_info: {
    "width": number (pixel)
    "height": number (pixel)
    "resolution": number (meters)
    "origin_x": number (meters)
    "origin_y": number (meters)
  }
  "map": string,
  "x": number (m),
  "y": number (m),
  "yaw": number (rad),
}
```

responses

- 200, Successful operation
- 400, Invalid map or waypoints format
- 500, Internal error

example value

- 500, "Error: did not write navigation map.yaml"

GET

parameters

- no parameters

request body

- no request body

responses

- 200, Successful operation
- 404, Image file not found
- 500, Connection failed

example value

```
{
```

```

"waypoints": [
  {
    "name": "1",
    "x": 1.04,
    "y": 4.23,
    "yaw": 0.5,
    "type": "target"
    "index": 0
  },
  ..... // other waypoints
],
"areas": [
  index: 0
  type: slowdown_areas,
  name: area_1
  points: [{x:0,y:0,index:0}...
]
]
"map": string,
"map_info": {
  "width": number (pixel)
  "height": number (pixel)
  "resolution": number (meters)
  "origin_x": number (meters)
  "origin_y": number (meters)
},
"x": number (m),
"y": number (m),
"yaw": number (rad),
}

```

/navigation-mode

Current navigation mode of the system: autonomous or manual

POST

parameters

- no parameters

request body

- type: string ["manual" | "auto" | "external"]
 - "manual" to use the GUI's virtual joystick
 - "auto" for moving the robot using Compass Base navigation stack
 - "external" to use a proprietary external joystick for moving the robot

example value

```

{
  "type": "manual"
}

```

responses

- 200, Successful operation
- 400, Bad request
- 500, Internal error

example value

- 200, "Success"
- 404, "ERROR: Navigation mode not found."

/obstacles

Get current obstacles vertex and related sensor in world frame

GET

parameters

- no parameters

request body

- no request body

responses

- 200, Successful operation
- 404, Image file not found
- 500, Connection failed

example value

```
{
  "content":
  [
    {
      "sensor": "laser",
      "vertex": [
        {
          "x": 1.04,
          "y": 4.23,
        }
        ...// other vertices
      ]
    },
    ..... // other obstacles
  ],
}
```

/raw-scans

Get the list of all the scans resulted from mapping operations

GET

parameters

- no parameters

request body

- no request body

responses

- 200, Successful operation
- 404, Raw scans folder not found

example value

```
{
```

```
{
  "content":
  [
    'rawscan1',
    'rawscan2',
    .....,
    'rawscanN'
  ]
}
```

/raw-scans/{name}

Get or delete a single scan map

GET

parameter

- "name"

request body

- no request body

response

- 200, successful operation
- 404, folder or png not found

example value

```
{
  "content":
  {
    "scan": string (image encoded in base64 format)
    "map_info": {
      "width": number (pixel)
      "height": number (pixel)
      "resolution": number (meters)
      "origin_x": number (meters)
      "origin_y": number (meters)
    }
    "x": number (m)
    "y": number (m)
    "yaw": number (rad)
  }
}
```

DELETE

parameter

- "name"

request body

- no request body

response

- 200, successful operation
- 404, scan map not found
- 500, not able to delete the scan map

example value

- 404, "Error: RawScan 20240512 folder not found"

- 200, "Success"

/robot-status

Get the current status of the robot

GET

parameters

- no parameters

request body

- no request body

responses

- 200, json formatted string

example value

```
{
  "content":
  {
    "status": string {IDLE, RUN, PAUSE, ERROR},
    "errors": [DRIVER_STUCK, LASER_ERROR, LOCALIZATION_JUMP, LOCALIZATION_TIMEOUT,
PATH_NOT_FOUND, PLC_CONNECTION_ERROR, ROBOT_OUT_OF_MAP, ROBOT_STUCK,
SAFETY_ERROR],
    "pose":
    {
      "x": number (meters),
      "y": number (meters),
      "yaw": number (radians)
    }
  }
}
```

Note: for a complete description about "status" and "errors" see the User Guide, SubSection "Robot Status"

/set-parameters

POST

parameters

- no parameters

request body

- params: array of objects
 - name: string {linear_velocity, linear_acceleration, angular_velocity, angular_acceleration, padding, obstacle_stop_distance}
 - value: number

example value

```
{
  "params": [
    {
      "name": linear_velocity,
      "value": 0.2
    },
  ],
}
```

```

        {
            "name": linear_acceleration,
            "value": 0.3
        }
    ]
}

```

response

- 200, successful operation
- 400, invalid request
- 500, unable to update parameters

example value

- 200, "Success"
- 500, "Unable to update planner parameters"

Note that this endpoint returns a valid response only in PreNavigation or Navigation status.

/system-status

Get or change the system modality.

POST

parameters

- no parameters

request body

- status: string [Idle | PreMapping | Mapping | PreNavigation | Navigation | Configuration | Test]

example value

```

{
    "status": "Idle"
}

```

responses (code, description)

- 200, successful group change
- 400, invalid group switch request
- 500, group switch not possible

It is not possible to ask a direct system status change from PreMapping to Mapping or from PreNavigation to Navigation by using a POST request on /system-status but rather the system status change is activated by using a specific action request. In particular:

- from PreMapping to Mapping (and vice versa) is possible through the POST request on /mapping as defined in the related section
- from PreNavigation to Navigation (and vice versa) is possible through the POST request on /mission as defined in the related section

GET

parameters

- no parameters

request body

- no request body

responses

- 200, successful operation
- 500, Connection to db failed: retry operation

example value

```

{

```

```
    "content": {
      "status": string [Idle | PreMapping | Mapping | PreNavigation | Navigation | Configuration | Test
| Transition ]
      "error": string
    }
  }
```

/system-status/{status}

GET

parameters

- status

request body

- no request body

responses

- 200, successful operation, the sub-status is returned
- 500, Connection to db failed: retry operation
- 400, Sub status does not exist

example values depend on the sub system we are checking. Each sub system has different fields in its response. You can check the fields by observing the following examples.

Note: state Transition has no sub state

Idle

```
{
  "content": {
    "configured_once": true
    "mapped_once": false
  }
}
```

Configuration

```
{
  "content": {
    "configured_once": false
  }
}
```

PreMapping or Mapping

```
{
  "content": {
    "navigation_mode": "manual"
  }
}
```

PreNavigation or Navigation

```
{
  "content":
  {
    "navigation_mode": "auto"
    "mission_pause": false,
    "safety_active": false
  }
}
```

Test

```
{
  "content":
    "tests": {
      {
        "name": [ping_pc, ping_plc, ping_sensors, check_scan, odometry_translation,
odometry_rotation, check_opc_variables] (Array of string)
        "result": [true,true,true,true,true] (Array of boolean)
        "resultMsg": ["Ok","Ok","Ok","Ok","Ok"] (Array of string)
        "distance": [0,0,0.99,3.14,0] (array of double)
        "playing": [0,0,0,1,0,0] Array of int)
      }
    }
}
```

Size of array depends on the system configuration

/system-status/substatus

GET

parameters

- status

request body

- no request body

responses

- 200, successful operation, the sub-status is returned
- 500, Connection to db failed: retry operation
- 400, Sub status does not exist

```
{
  "content":
  {
    "configured_once": true
    "mapped_once": false
    "navigation_mode": "auto"
    "mission_pause": false,
    "safety_active": false,
    "tests": {
      "name": [ping_pc, ping_plc, ping_sensors, check_scan, odometry_translation,
odometry_rotation, check_opc_variables] (Array of string)
      "result": [true,true,true,true,true] (Array of boolean)
```

```

        "resultMsg": ["Ok","Ok","Ok","Ok","Ok"] (Array of string)
        "distance": [0,0,0.99,3.14,0] (array of double)
        "playing": [0,0,0,1,0,0] (Array of int)
    }
}

```

/software-version

Read the software version from file

GET

parameters

- no parameters

request body

- no request body

responses

- 200, Success
- 404, File not found
- 400, Error in opening the file

/test

POST

parameters

- no parameters

request body

- test: string {ping_pc, ping_plc, ping_sensors, check_scan, odometry_translation, odometry_rotation, check_opc_variables}
- action: string {start, stop} - odometry only
- distance: number (m | rad) - odometry start only

example value

```

{
    "test": "odometry_translation",
    "action": "start",
    "distance": 1.5
}

```

response

- 200, successful operation
- 400, Invalid request
- 500, a test failed to start/stop

example value

- 200, "Success"
- 500, "Unable to stop odometry test"

/velocities

Get the current linear and angular velocities of the robot

GET

parameters

- no parameters

request body

- no request body

responses

- 200, json formatted string

example value

```
{
  "content": {
    "requested": {
      "linear": {
        "x": 0.0,
        "y": 0.0
      },
      "angular": {
        "z": 0.0
      }
    },
    "measured": {
      "linear": {
        "x": 0.0,
        "y": 0.0
      },
      "angular": {
        "z": 0.0
      }
    }
  }
}
```

/world-pose

Take the robot position in world frame

GET

parameters

- no parameters

request body

- no request body

responses

- 200, json formatted string
- 404, Failed to open png

example value

```
{
  "content":
```

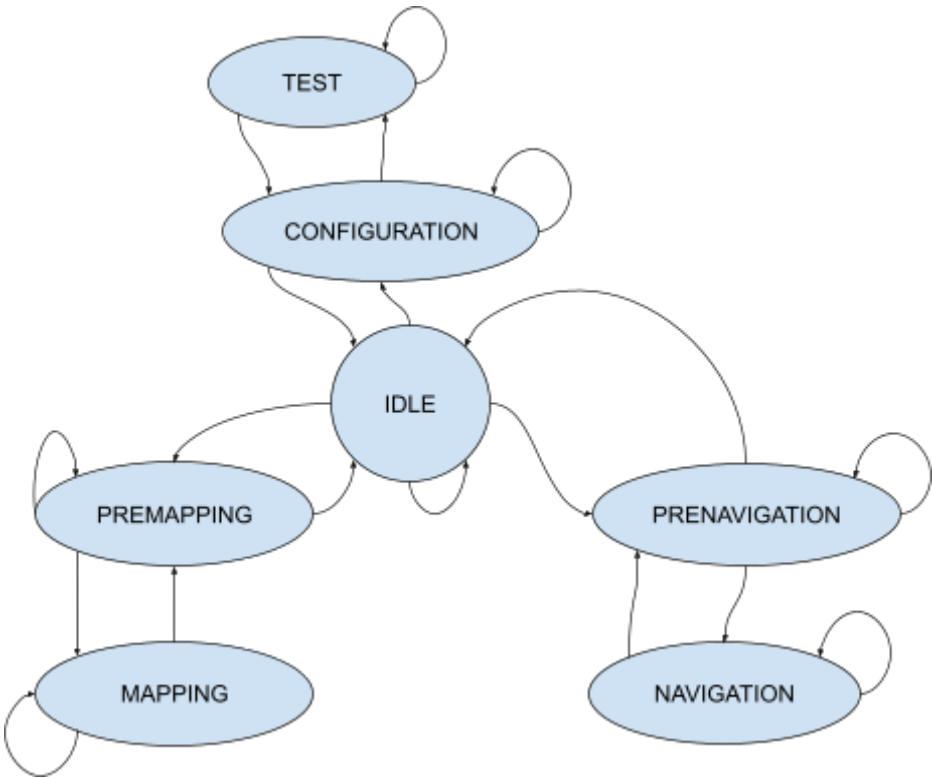
```
{
  "pose":
  {
    "x": number (meters)
    "y": number (meters)
    "yaw": number (radians)
  }
  "map_info":
  {
    "width": number (pixel)
    "height": number (pixel)
    "resolution": number (adimensional)
    "origin_x": number (meters)
    "origin_y": number (meters)
  }
}
```

Appendix

State machine

APIs related to specific endpoints can be used only when the right system status is on. When a system status change is triggered, the system tries to turn on the related processes. if these ones cannot be turned on, the system changes back to the previous state.

The following image represents the state machine of Proxima Compass Base.



The table below is the possible state transition between a state and another.

	IDLE	PREMAPPING	MAPPING	PRENAVIGATION	NAVIGATION	CONFIGURATION	TEST
IDLE	✓	✓		✓		✓	
PREMAPPING	✓	✓	✓				
MAPPING		✓	✓				
PRENAVIGATION	✓			✓	✓		
NAVIGATION				✓	✓		
CONFIGURATION	✓					✓	✓
TEST						✓	✓

Please note state change is not always direct. The following table shows in which state change we have also an intermediate state called Transition the user should be aware of:

	IDLE	PREMAPPING	MAPPING	PRENAVIGATION	NAVIGATION	CONFIGURATION	TEST
IDLE		✓		✓			
PREMAPPING			✓				
MAPPING							
PRENAVIGATION							
NAVIGATION							
CONFIGURATION							✓
TEST							

The table has to be read as: from state in row to state in column we have an intermediate state 'Transition'. If transition from one state to another fails, the state machine returns to the state before transition starts.

Endpoints vs State

The following table shows in which system mode every endpoint should be used. If these constraints are not satisfied, the behavior of the system is undefined.

Endpoint (for POST or GET operation see the endpoint specific API)	Home	Configure	PreMapping/Mapping	Test	PreNavigation/Navigation
/configuration		✓			
/custom-waypoint-mission					✓
/global-path					✓
/joy-input			✓		
/laser/{step}					✓
/localize-pose					✓
/localize-waypoint					✓
/login	✓	✓	✓	✓	✓
/logout	✓	✓	✓	✓	✓
/map-info			✓		✓
/mapping			✓		
/mission					✓
/move					✓
/navigation-map			✓		✓
/navigation-mode					✓
/obstacles					✓

/map-pose			✓		✓
/raw-scans			✓		
/raw-scans/{name}			✓		
/robot-status					✓
/set-parameters					✓
/software-version	✓	✓	✓	✓	✓
/system-status	✓	✓	✓	✓	✓
/system-status/{status}	✓	✓	✓	✓	✓
/system-status/substatus	✓	✓	✓	✓	✓
/test				✓	
/velocities					✓
/world-pose					✓

Token and timeout

The token generated from the login procedure is valid for **1 minute**; if no requests are made to the system within this period, the token expires. However, there two different scenarios:

Web app scenario

- When a valid login procedure is accomplished, a token is stored in the local storage of the web browser. This token will be automatically attached to each following backend request in the *Authorization* header field. Since the web app executes a GET request to /system-status every 5 seconds, the user doesn't have to worry about it.

REST API only scenario

- In this case, when a valid login is accomplished, the user has to take care manually of the resulting token. The user is also in charge of manually attaching the token to the successive requests in the *Authorization* header field. Remember that if no requests have been made in 1 minute, the token expires and a new login is necessary.

Test and configuration combinations

The number of possible tests to get and to launch depends on the system configuration previously saved.

The following table represents the feasible tests to launch in the POST operation of endpoint /test and the possible test to retrieve in a GET operation on the endpoint /system-status/Test:

	Opcua protocol	No opcua protocol
Ros2 sensor	"ping_plc", "check_opc_variables", "ping_sensors"	"ping_sensors"
No ros2 sensor	"ping_plc", "check_opc_variables"	

Following tests are always feasible for POST and present in GET:

- "ping_pc",
- "check_scan",
- "odometry_rotation",

- "odometry_translation"